# A3-PS001-TC001

Android Ransomware-like Activity Early Warning in a Controlled Lab

**Jelish G & Swetha C**

jelishg26@karunya.edu.in

swethac@karunya.edu.in

AIORI Hackathon 3

October 14, 2026

# Problem Description

**Android ransomware is usually caught only after files are already encrypted or the device is locked.**

- Crypto-ransomware reads, encrypts and renames user files within seconds; signature-based scanners react late and miss repacked or obfuscated variants.
- Users have little visibility and no easy rollback once files on shared storage are overwritten.
- Need: an early-warning signal from runtime behaviour (bursty file writes, mass renames, entropy jump) that fires before most files are lost.
- **Scope:** a lightweight behavioural detector built and validated entirely in an isolated emulator lab, using a harmless simulator and dummy files. No real malware is used.

# Background

**Two families dominate mobile ransomware, and they leave different runtime traces.** [1]

- **Locker:** abuses overlays, device-admin or accessibility services to block the screen, with no file damage.
- **Crypto:** enumerates shared storage, then encrypts, overwrites or renames files and drops a ransom note.
- Behavioural fingerprint: burst of file modifications, extension changes, high-entropy content and repeated note files [5] [7].
- Scoped storage and runtime permissions reduce exposure but do not remove it (e.g., broad file-access permissions, older target SDKs).
- A controlled lab (offline emulator, dummy data, snapshot reset) lets us study these traces safely and repeatably.

# Literature & RFC Review

**Most prior work is static analysis or desktop file-system monitoring; few give an early, on-device behavioural signal.**

- **HelDroid** [1]: static and heuristic detection of locker and crypto traits.
- **Song et al.** [2]: process monitoring on Android to detect and prevent ransomware.
- **R-PackDroid** [3] **and Alsoghyer & Almomani** [4]: machine-learning classifiers over API-package and app features.
- **CryptoLock** [5] **and ShieldFS** [6] (desktop): file-activity indicators such as entropy change and bulk renames; they inspire our features.
- **Gap:** reproducible, early-warning evaluation of file-behaviour signals on Android in a safe lab.

# Proposed Solution

**A behaviour-based early-warning pipeline built in an isolated Android emulator lab.**

1. **Lab:** Android Studio emulator, network off, clean snapshot, about 100 dummy files in /sdcard/TestLab.
2. **Workload:** a harmless simulator app that renames and rewrites only the dummy files, plus benign apps (gallery, backup, zip) as negative cases.
3. **Capture:** timestamped create, modify, rename and delete events via FileObserver and adb.
4. **Detect:** 1 to 5 second sliding windows score change rate, renames, entropy jump and note-file drop into a risk score (optional ML layer).
5. **Alert:** on-device notification, alert log and dashboard timeline; reset the snapshot and repeat.

# Optimization Proposed

## Optimizations over a single fixed-threshold baseline

| Aspect | Baseline | Optimized |
|---|---|---|
| **Detection logic** | One threshold on file-change rate | Weighted risk score over rate, renames, entropy and note drop |
| **Time window** | Single long window | Short sliding windows (1 to 5 s) for earlier alerts |
| **Entropy cost** | Scan whole files | Sample the first few KB of each file for low overhead |
| **False positives** | Backup and zip apps get flagged | Tuned against benign-app baselines |
| **Learning layer** | None | Optional Random Forest or Isolation Forest on window features |

*Time-to-detect, files affected and false-positive rate are measured in the evaluation phase.*

# Solution Architecture & Design

**Five-stage pipeline across an isolated emulator and a host PC.**

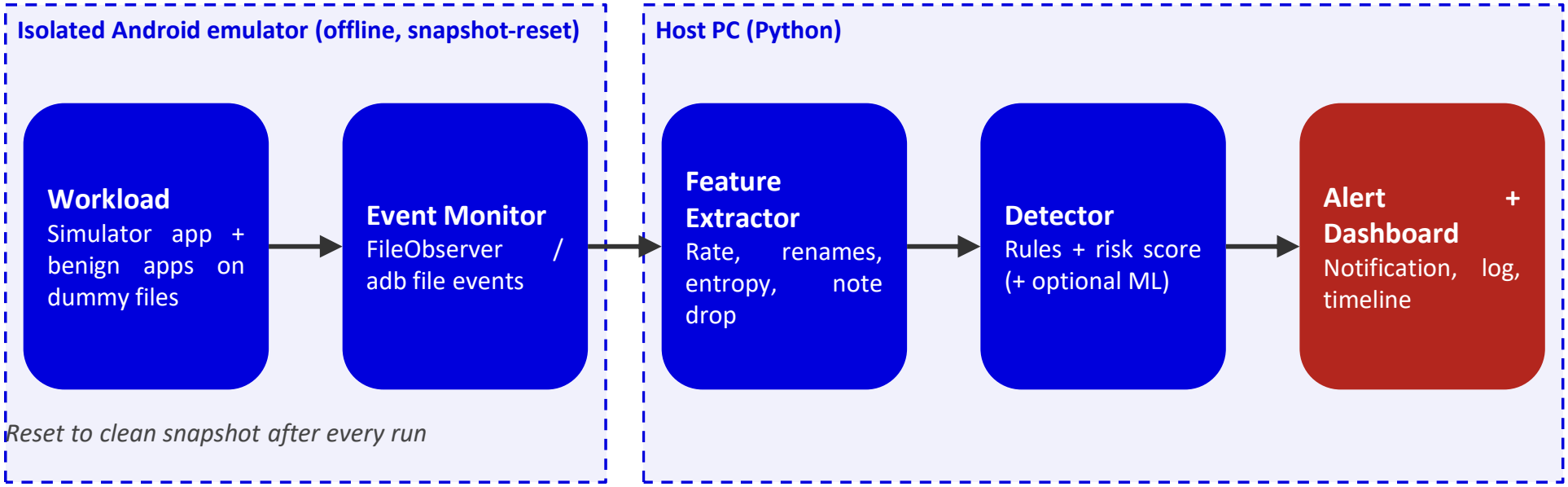



Figure: System architecture of the early-warning lab

# Data Flow Diagram

**Level-1 flow: file events to features, risk score and alerts.**

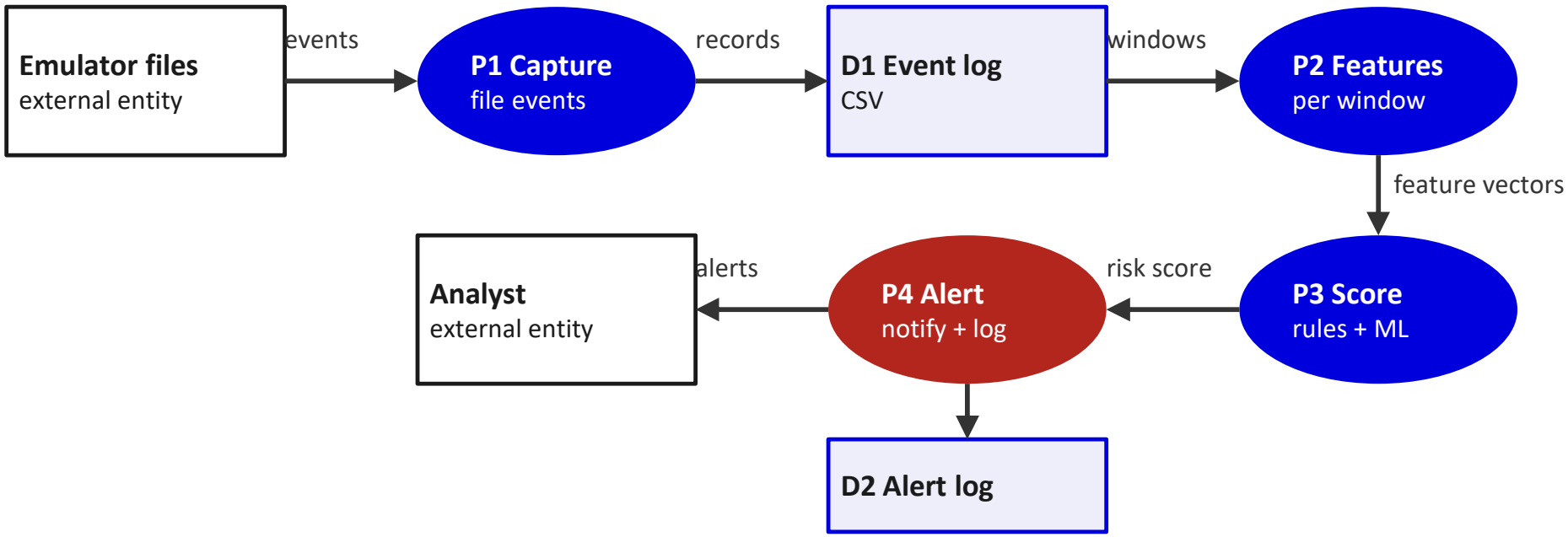
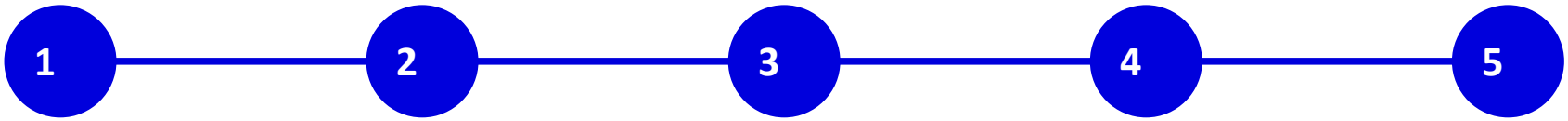



Figure: Data Flow Diagram (DFD)

# Deployment Timeline

**Five phases, each ending in a testable deliverable.**

**Method:** build, run in the emulator, reset the snapshot, tune thresholds, repeat.




| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| **Lab setup** | **Workload** | **Detection** | **Alerting** | **Evaluation** |
| Emulator, snapshot, dummy files | Simulator, benign apps, event capture | Rules, risk score, optional ML | Notification and dashboard | Repeated runs, metrics, slides |
| **Reusable lab** | **Labelled event data** | **Working detector** | **Live demo** | **Results + demo video** |

# Bibliography




1. N. Andronio, S. Zanero, F. Maggi. “HelDroid: Dissecting and Detecting Mobile Ransomware.” In: RAID 2015.
2. S. Song, B. Kim, S. Lee. “The Effective Ransomware Prevention Technique Using Process Monitoring on Android Platform.” In: Mobile Information Systems (2016).
3. D. Maiorca et al. “R-PackDroid: API Package-Based Characterization and Detection of Mobile Ransomware.” In: ACM SAC (2017).
4. S. Alsoghyer, I. Almomani. “Ransomware Detection System for Android Applications.” In: Electronics 8.8 (2019), 868.
5. N. Scaife et al. “CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data.” In: IEEE ICDCS (2016).
6. A. Continella et al. “ShieldFS: A Self-healing, Ransomware-aware Filesystem.” In: ACSAC (2016).
7. MITRE. “ATT&CK for Mobile: Data Encrypted for Impact (T1471).” attack.mitre.org.

Thank You for Your Attention!